

University of Sousse

National Engineering School of Sousse



Module Project Report : Cultural Algorithms

Realized by:

Oussema Ben Ameer Yacin Ben Kacem

Supervised by :

Dr. Walid Chainbi

December 2024

Contents

1	Introduction to Cultural Algorithms	2
1.1	Fundamental Principles	2
1.2	General Architecture	2
1.3	Types of Knowledge in the Belief Space	2
2	Classical Example: Function Optimization	3
2.1	Minimization Problem	3
2.2	Cultural Algorithm Application	3
3	Project Implementation: Cultural Wordle Solver	3
3.1	Project Context	3
3.2	Cultural Algorithm Adaptation	3
3.2.1	Belief Space Structure	3
3.2.2	Constraint System	4
3.2.3	Population and Evolution	4
3.3	Main Function	4
3.4	Resolution Algorithm	5
3.5	Execution Example	5
3.6	Advantages of the Cultural Approach	6
4	Conclusion	6

1 Introduction to Cultural Algorithms

Cultural Algorithms (CA) are an optimization metaheuristic inspired by human cultural evolution. Proposed by Robert Reynolds in 1994, they are based on the principle that human societal evolution depends not only on genetic transmission but also on cultural knowledge transmission.

1.1 Fundamental Principles

A cultural algorithm relies on two main interconnected components:

- **Population Space:** Represents individuals exploring the search space. Similar to genetic algorithms, this population evolves through selection, crossover, and mutation operations.
- **Belief Space:** Constitutes the system's collective memory, storing accumulated knowledge across generations. This space guides population evolution by influencing individuals.

1.2 General Architecture

The execution cycle of a cultural algorithm follows these steps:

1. **Initialization:** Creation of an initial population and empty belief space
2. **Evaluation:** Calculation of each individual's performance
3. **Acceptance:** Selection of best individuals to update the belief space
4. **Influence:** Using knowledge to guide creation of new individuals
5. **Evolution:** Application of classic evolutionary operations
6. **Iteration:** Repetition until stopping criterion is satisfied

1.3 Types of Knowledge in the Belief Space

The belief space can contain different types of knowledge:

- **Normative Knowledge:** Defines acceptable value ranges for each dimension of the search space
- **Situational Knowledge:** Stores the best solutions found and their contexts
- **Topographic Knowledge:** Maps promising regions of the search space
- **Historical Knowledge:** Maintains history of explored solutions
- **Domain Knowledge:** Problem-specific rules

2 Classical Example: Function Optimization

2.1 Minimization Problem

Consider a simple optimization example: finding the minimum of the Rastrigin function in 2 dimensions:

$$f(x, y) = 20 + x^2 + y^2 - 10(\cos(2\pi x) + \cos(2\pi y))$$

with $x, y \in [-5.12, 5.12]$. The global minimum is $f(0, 0) = 0$.

2.2 Cultural Algorithm Application

Population: Set of points (x, y) in the search space.

Normative belief space: For each dimension, we store intervals $[x_{min}, x_{max}]$ and $[y_{min}, y_{max}]$ corresponding to the best solutions.

Influence process: New individuals are preferentially generated in regions defined by normative knowledge.

Evolution: Application of mutations and crossovers to explore the space.

Over iterations, the belief space narrows around the optimal point $(0, 0)$, efficiently guiding the search.

3 Project Implementation: Cultural Wordle Solver

3.1 Project Context

The Wordle game consists of guessing a 5-letter word in a maximum of 6 attempts. After each guess, the game provides colored feedback:

- **Green (2):** Letter is correct and in the right position
- **Yellow (1):** Letter is in the word but in wrong position
- **Gray (0):** Letter is not in the word

3.2 Cultural Algorithm Adaptation

My project adapts cultural algorithms to automatically solve Wordle. The implementation combines evolutionary exploration and exploitation of accumulated knowledge.

3.2.1 Belief Space Structure

The belief space is structured into two components:

```
1 belief_space = {
2     'normative': [{_ for _ in range(WORD_LENGTH)],
3     'situational': {
4         'best_word': None,
5         'best_score': -1.0
6     }
7 }
```

Normative Knowledge: For each position (0 to 4), we store frequencies of letters observed in correct words. This guides selection toward words with statistically promising letters at each position.

Situational Knowledge: Memorizes the best candidate word and its heuristic score.

3.2.2 Constraint System

A constraint system captures information obtained from Wordle feedback:

```
1 constraints = {  
2     'negation': set(), # Absent letters (gray)  
3     'mandatory': defaultdict(lambda: {  
4         'must_include': 0, # Letter present  
5         'must_not_be': set() # Forbidden positions  
6     })  
7 }
```

- **Negation:** Set of gray letters (absent from the word)
- **Mandatory:** For each yellow or green letter, indicates it must be included and positions where it cannot be

3.2.3 Population and Evolution

The population consists of candidate words from the dictionary. It evolves according to:

1. **Filtering:** After each attempt, the dictionary is filtered to keep only words compatible with obtained feedback
2. **Influence:** Individuals are selected and potentially modified according to belief space knowledge
3. **Evolution:** Application of evolutionary operations to maintain diversity

3.3 Main Function

The main function `interactive_cultural_wordle` orchestrates the process:

```
1 def interactive_cultural_wordle(  
2     target_word,  
3     initial_words,  
4     pop_size,  
5     acceptance_ratio,  
6     influence_ratio,  
7     max_guesses=6  
8 ):
```

Parameters:

- `target_word`: The target word to guess
- `initial_words`: Dictionary of valid words

- `pop_size`: Population size (50 in our case)
- `acceptance_ratio`: Proportion of individuals accepted to update beliefs (0.2)
- `influence_ratio`: Intensity of belief influence (0.3)
- `max_guesses`: Maximum number of attempts (6)

3.4 Resolution Algorithm

At each iteration (attempt):

1. **Heuristic Selection:** The function `get_best_heuristic_guess` chooses the best candidate word by combining:
 - Information from the belief space
 - Accumulated constraints
 - A heuristic score favoring discriminating words
2. **Feedback Acquisition:** The word is tested via `get_wordle_pattern`, returning a tuple of patterns (0, 1, 2)
3. **Constraint Update:** According to the received pattern:

```

1 for i, (char, p) in enumerate(zip(best_guess,
2     feedback_pattern)):
3     if p == 0: # Gray
4         if char not in target_word:
5             constraints['negation'].add(char)
6     elif p == 1: # Yellow
7         constraints['mandatory'][char]['must_include'] = 1
8         constraints['mandatory'][char]['must_not_be'].add(i)
9     elif p == 2: # Green
10        constraints['mandatory'][char]['must_include'] = 1

```

4. **Dictionary Filtering:** Elimination of incompatible words
5. **Belief Space Update:** Enrichment with new valid words
6. **Influence and Evolution:** Population guidance according to beliefs

3.5 Execution Example

For the target word CROWN, the algorithm produces:

TARGET WORD IS: CROWN

Starting Interactive Cultural Wordle Solver.

Guess 1: SLATE ->

(Remaining candidates: 842)

Guess 2: CROWN ->

Solver Found the word!

Game finished in 2 guesses.

3.6 Advantages of the Cultural Approach

Using a cultural algorithm for Wordle offers several advantages:

- **Cumulative Learning:** The belief space accumulates knowledge about letter distribution
- **Exploration/Exploitation Balance:** The population explores the solution space while beliefs exploit the best regions
- **Adaptability:** The system dynamically adapts to received feedback
- **Robustness:** The combination of constraints + heuristics + evolution ensures rapid convergence

4 Conclusion

Cultural algorithms offer a powerful framework for optimization problems by combining population evolution and knowledge transmission. This project demonstrates their adaptability to unconventional domains such as word game solving.

The Cultural Wordle Solver implementation illustrates how cultural evolution principles can be translated into efficient computational mechanisms: the belief space acts as collective memory, constraints capture game rules, and the population intelligently explores the solution space.

Results show that this hybrid approach typically solves Wordle in 3-4 attempts on average, which is comparable to expert human performance.

References

- Reynolds, R. G. (1994). An introduction to cultural algorithms. In *Proceedings of the Third Annual Conference on Evolutionary Programming*.
- Jin, X., & Reynolds, R. G. (1999). Using knowledge-based evolutionary computation to solve nonlinear constraint optimization problems. *Journal of Artificial Intelligence Research*.